Dokumentasjon for sudokusolver

Jone Bastiansen

April 2025

1 Introduksjon

Dette programmet lar brukeren sende inn en tekstfil med et sudokubrett på formen som vist i bildet under (figur 1). Brukeren kan velge om hen vil bare få løst brettet kjapt eller om hen vil se en visualisering av løsningsprossesen i et AnimationWindow.

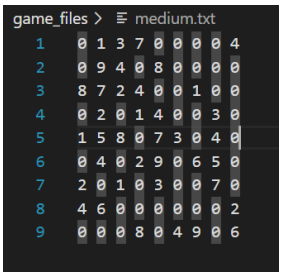


Figure 1: Formatet på tekstfilene

2 Metode for løsning

Programmet lagrer informasjon om hver celle i en klasse som heter Block. I Block-klassen har vi et set av muligheter som ligger i possibleNumbers variabelen. Når et sudokubrett blir initialisert starter alle cellene som ikke er løst fra før av med alle tall fra 0-9 som muligheter. Metoden for å gjøre settet av løsninger for hver block mindre ligger for det meste i alle funksjonen i improveCluster.h. En "Cluster" er en vector med pekere til blocks. Det kan være en rad, en kolonne eller en 3x3 boks i brettet. Måten funksjonene i improveCluster begrenser mulighetene er ved å se på alle mulige subsets på en viss størrelse av blocker i clusteren. Dermed kan programmet se på unionen av alle mulighetene i blockene i subsetet og dersom størrelsen på unionen av muligheter er like stor som størrelsen på subsetet kan programmet slette de tallene fra mulighetene i alle andre blockene.

Denne metoden er ikke perfekt og det finnes brett som den ikke greier å løse. Når jeg har testet den på brett fra sudoku.com så har den funket på alle

vanskelighetsgradene utenom den alle vanskeligste, extreme. Jeg hadde egentlig tenkt til å implementere funksjonalitet som gjorde at når solveren ble stuck skulle den gjette en løsning og fortsette prosessen og backtracke dersom det viste seg at den løsningen var feil. Dette fikk jeg ikke tid til.

3 Solver klassen og EventListener

Solver klassen er den som sørger for at funksjonene i improveCluster bli applisert riktig på sudokubrettet. I tillegg til å inneholde en peker til brettet inneholder den også en vector med pekere til eventListeners. Det finnes to typer subklasser av eventListener; GraphicEventListener og LogginEventListener. Disse to inneholder de samme funksjonene, men gjør forskjellige ting. LoggingEventListener skriver til terminalen, mens GraphicEventListener tegner brettet i en AnimationWindow og viser løsningsprosessen. Gjennom løsningsprosessen kaller solveren på disse funksjonene med å bruke std::for_each.

4 Hjelpemidler

Jeg har brukt en rekke nettsider for å hjelpe meg med prosjektet. De har hovedsakelig bare hjulpet meg med å finne funksjoner som var nyttige som jeg ikke visste fantes eller som jeg hadde glemt hvordan jeg brukte. Et eksempel på en slik funksjon er std::for_each. Nettsidene jeg brukte var stackoverflow.com, cplusplus.com, cppreference.com og w3schools.com.

Jeg brukte også GitHub Copilot. Copilot ble mest brukt til å hjelpe meg med debugging og for å finne småfeil. Det er to funksjoner i koden som Copilot er alene ansvarlig for å ha laget. Disse to funksjonen er generateSubsets og getSubsets. Disse to funksjonene samme tar inn en cluster og returnerer alle mulige subsets av en viss størrelse. De bruker en slags rekursjon som jeg syntes var vanskelig å forstå.

5 Mangler

Det er noen ting som var planlagt å ha med i programmet som jeg ikke fikk tid til å implementere.

- Gjetting av løsning hvis programmet setter seg fast
- Skriving til fil etter at brett er løst
- Grafisk brukergrensesnitt
- Generell rydding og optimalisering av kode
- Sjekke om formatet på filen er riktig

6 Refleksjon

Under utviklingen av prosjektet fokuserte jeg aller mest på algoritmen som løser brettet og visualiseringen av løsningsprosessen. Jeg fokuserte lite på brukervennlighet og feilhåndtering i innlesing av filer. Hvis jeg hadde hatt mer tid så ville jeg nok også fokusert litt på disse tingene. Under utviklingsprosessen tok jeg noen valg jeg er fornøyd med og noen valg jeg er mindre fornøyd med. Blant valgene jeg er fornøyd med er:

- Skille funksjonaliteten til solveren med funksjonalitet som tegner til AnimationWindow ved å bruke Eventlistener.
- Hvordan jeg implementerte GraphicEventlistener ved å lagre informasjon om forskjellige farger og størrelser på ting i medlemsvariabler.
- Klassen Solver til å løse brettet.
- Bruk av pekere

Valg som jeg ikke er like fornøyd med:

- Bruk av std::set¡char¿ for possibleNumber is Block klassen. Tanken var at char bruker mindre minne, men det bare kompliserte ting og var ikke nødvendig.
- I litt latskap så brukte jeg litt mye funksjoner som ikke var medlem i noe klasse, hovedsakelig i ImproveCluster. Her kunne jeg laget en klasse for Cluster.